Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3001B. Fundamentación de robótica (Gpo 101)

Actividad 2.2 PWM con ESP32

Alumnos:

- Bruno Alejandro Domínguez Maldonado | A01738844
- Iván Ariel Rebollar León | A01737891
- Omar Kuri Vázquez | A01738129

03 de marzo de 2026

Resumen

Este proyecto detalla la transición de un sistema de control local a una arquitectura distribuida utilizando ROS 2 y micro-ROS para el manejo de un motor de corriente continua (DC), se implementó un nodo en un microcontrolador ESP32 capaz de procesar comandos remotos para la dirección y, simultáneamente, realizar la lectura de un potenciómetro para calcular el ciclo de trabajo (PWM), el proceso abarca desde la configuración del agente en Linux hasta la publicación de variables críticas como voltaje, duty cycle y estado del motor en la red de ROS 2 y se resolvieron conflictos de asignación de pines y errores de resolución ADC, logrando una integración fluida que permite la visualización en tiempo real mediante herramientas de telemetría como rqt_plot.

Objetivos

Diseñar e implementar un sistema de control distribuido mediante micro-ROS que permita regular la velocidad y el sentido de giro de un motor DC, integrando la lectura de sensores analógicos y la publicación de datos de rendimiento en el ecosistema ROS 2.

- Configurar el entorno de trabajo (uros_ws) y el agente de micro-ROS para la comunicación serial a 115200 baudios.
- Migrar la lógica de control serial convencional a una estructura de Publishers (voltaje, duty cycle, estado) y Subscribers (comandos de dirección).
- Corregir errores de resolución en la conversión ADC-PWM para eliminar señales erráticas en el osciloscopio.
- Validar la comunicación bidireccional mediante el envío de comandos "D", "I" y "S" desde la terminal de Ubuntu.

Introducción

El control de actuadores en robótica moderna requiere una comunicación fluida entre el hardware de bajo nivel y los algoritmos de alto nivel, para ello, se utiliza micro-ROS, una capa de abstracción que permite a microcontroladores como el ESP32 participar en una red ROS 2 como nodos independientes.

El sistema se basa en tres pilares tecnológicos:

- PWM (Modulación por Ancho de Pulso): Técnica para controlar la potencia entregada al motor variando el ciclo de trabajo de una señal cuadrada. En este proyecto se ajustó a 500 Hz para mejorar el torque de arranque y reducir el zumbido mecánico.
- Circuito que permite invertir la polaridad del voltaje aplicado al motor, utiliza señales lógicas (IN3, IN4) para el sentido de giro y una señal de habilitación (ENB) para la velocidad vía PWM.
- Esquema Cliente-Agente: El ESP32 actúa como cliente, enviando y recibiendo datos a través de un "Agente" que corre en la computadora, el cual traduce los mensajes seriales al protocolo DDS de ROS 2.

Solución del problema

La metodología se centró en la integración de la lectura analógica y el control de suscripción. Se configuró un espacio de trabajo en Linux para habilitar el puente de comunicación bidireccional mediante el comando ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0.

- Identificación de Variables y Corrección del Error

  el error El error identificado residía en la discrepancia de resolución entre la entrada y la salida del microcontrolador, originalmente, el código utilizaba: pwm = map(pot, 0, 1023, 0, 255), el ESP32 tiene un ADC de 12 bits (rango 0–4095) y al asumir un máximo de 1023, cualquier giro del potenciómetro más allá del 25% saturaba el registro de 8 bits del PWM, generando una señal deformada o constante en el osciloscopio.

- Corrección del Error

  Ajuste de ADC, se reconoció la resolución de 12 bits del ESP32, utilizando el rango correcto de 0 a 4095 para el cálculo del voltaje y el mapeo y el mapeo correcto, se actualizó a map(pot, 0, 4095, 0, 255), asegurando que el 100% de la posición física del potenciómetro corresponda exactamente al 100% de la potencia del PWM.

- Generación de PWM (LEDC en ESP32)

  A diferencia de Arduino Uno, el ESP32 no usa analogWrite y utiliza un periférico llamado LEDC, primero en PWM_FREQ (500 Hz) definimos cuántas veces por segundo se repite el ciclo, bajamos la frecuencia de 5000Hz a 500Hz para que el motor tenga más "torque" en velocidades bajas, permitiendo que el campo magnético venza la fricción mecánica sin generar solo zumbido y enPWM_RES (8 bits): Configuramos la resolución del ciclo de trabajo de 0 a 255 niveles.

- Arquitectura micro-ROS: El Cliente y el Agente

  En este código, el ESP32 no es un simple dispositivo serial; es un Nodo de ROS 2, el Executor (rclc_executor_spin_some), es el "cerebro" que gestiona el tiempo y se encarga de revisar si ha llegado un comando por el suscriptor o si es hora de que el timer dispare la publicación de datos, por otro lado la gestión de memoria dinámica, los mensajes de tipo String en micro-ROS no tienen un tamaño predefinido, por eso usamos malloc para reservar espacio en la RAM del ESP32: msg_state.data.data = (char*) malloc(msg_state.data.capacity); sin esto, el programa intentaría escribir en una dirección de memoria nula y el ESP32 se reiniciaría infinitamente.

- Lógica de Control de Dirección (Puente H)

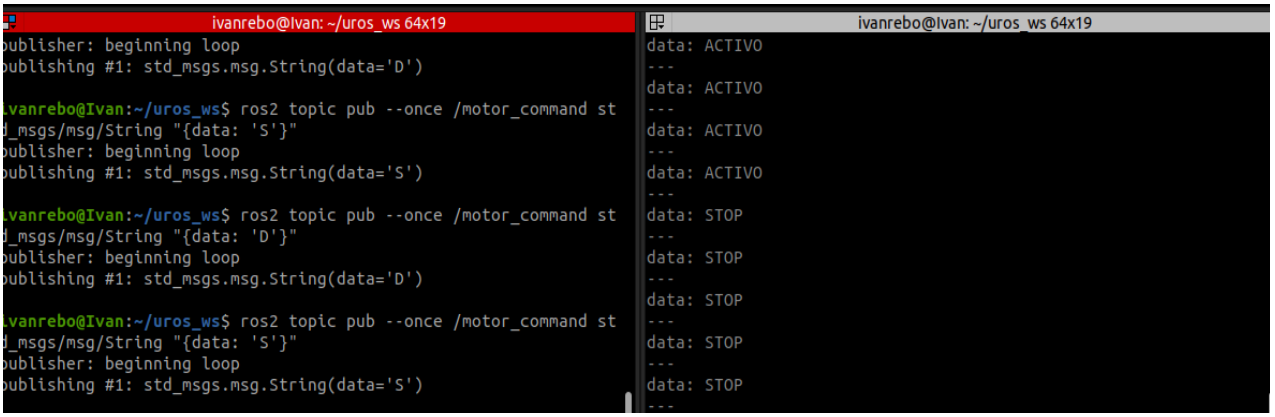
  El motor se controla mediante una lógica de conmutación en los pines 14 (In3) y 27 (In4). El pin 26 (ENB) actúa como la "llave de paso" (PWM).

  - Comando 'D': In3=HIGH, In4=LOW. La corriente fluye en un sentido.
  - Comando 'I': In3=LOW, In4=HIGH. La corriente se invierte.
  - Comando 'S': In3=LOW, In4=LOW. Se cortocircuitan las bobinas del motor para un frenado rápido.

Resultados

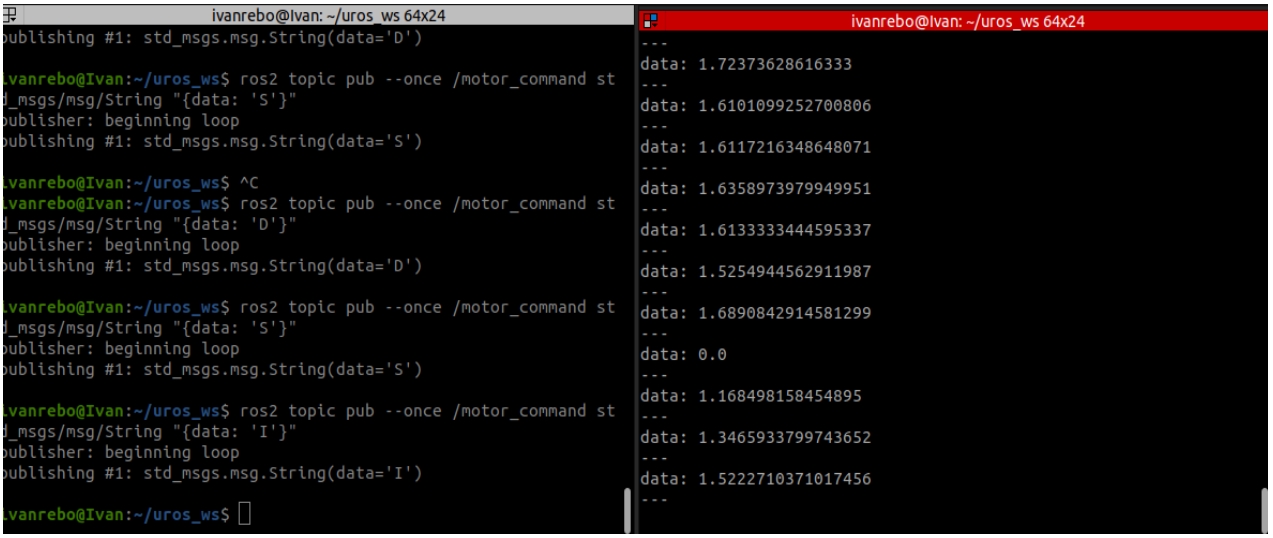
Publicar por el tópico la dirección y ver el estado del motor ("Activo a la izquierda o derecha", por ejemplo)

- Detener el motor y ver el estado "Detenido"



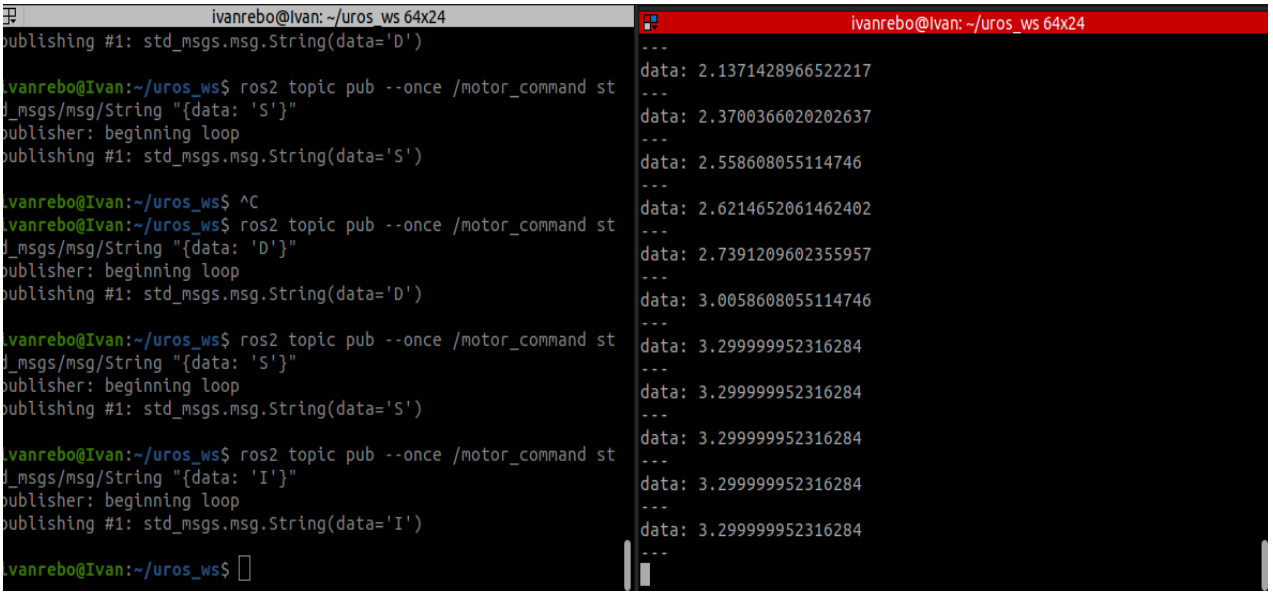
En la terminal izquierda se envía el comando "S" al tópico /motor_command utilizando ros2 topic pub --once, lo que indica la instrucción de detener el motor. En la terminal derecha se observa la publicación continua del estado del sistema, donde inicialmente aparece “ACTIVO” y posteriormente cambia a “STOP”. Este cambio confirma que el ESP32 recibió correctamente el comando y ejecutó la lógica de frenado del motor.

- Graficar el duty cycle del pwm:



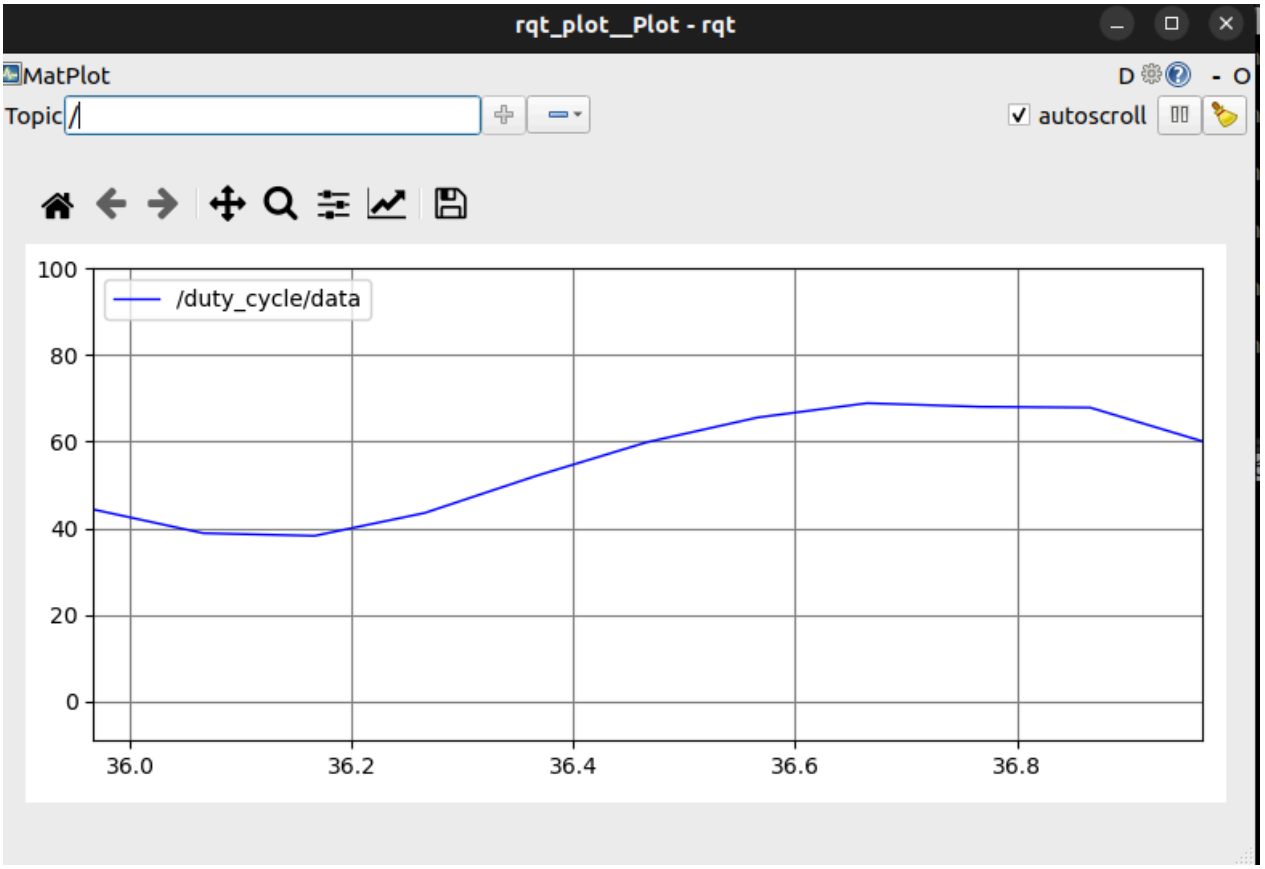
En la terminal izquierda se mandan los comandos "D", "S" e "I" al tópico /motor_command con pub --once para ir cambiando la dirección del motor. Mientras tanto, en la terminal de la derecha van saliendo los valores del duty cycle del PWM. Estos datos vienen de la lectura del potenciómetro y sirven para graficar después cómo se comporta la velocidad del motor.

- Graficar el duty cycle del pwm: (valor máximo)



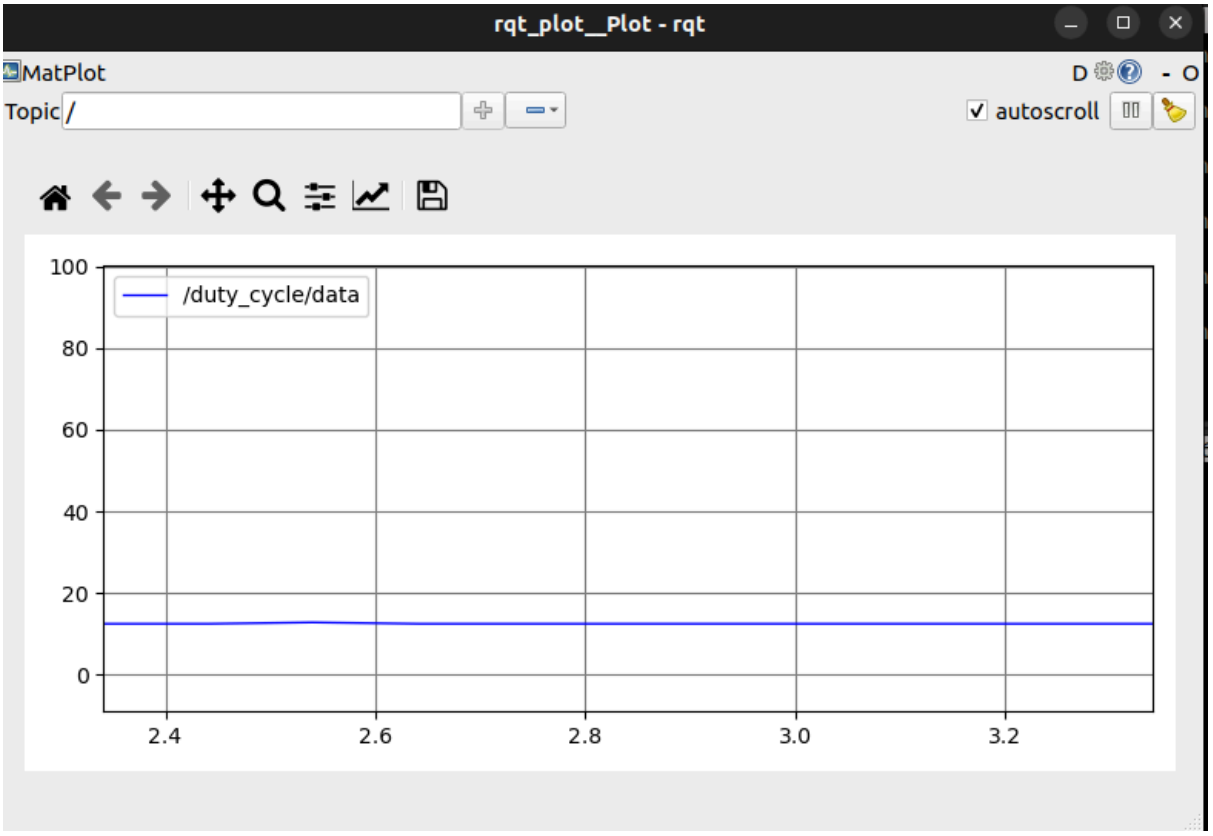
En la terminal izquierda se mandan comandos como 'D', 'S' e 'I' al tópico /motor_command con pub --once para controlar el motor. Mientras tanto, en la terminal derecha se ve cómo el duty cycle del PWM va subiendo hasta llegar a su valor máximo (que corresponde a los 3.3V de salida del ESP32). Esto confirma que el potenciómetro está mapeando bien todo el rango del ciclo de trabajo.

- Gráfica RQT plot (moviendo el potenciómetro)



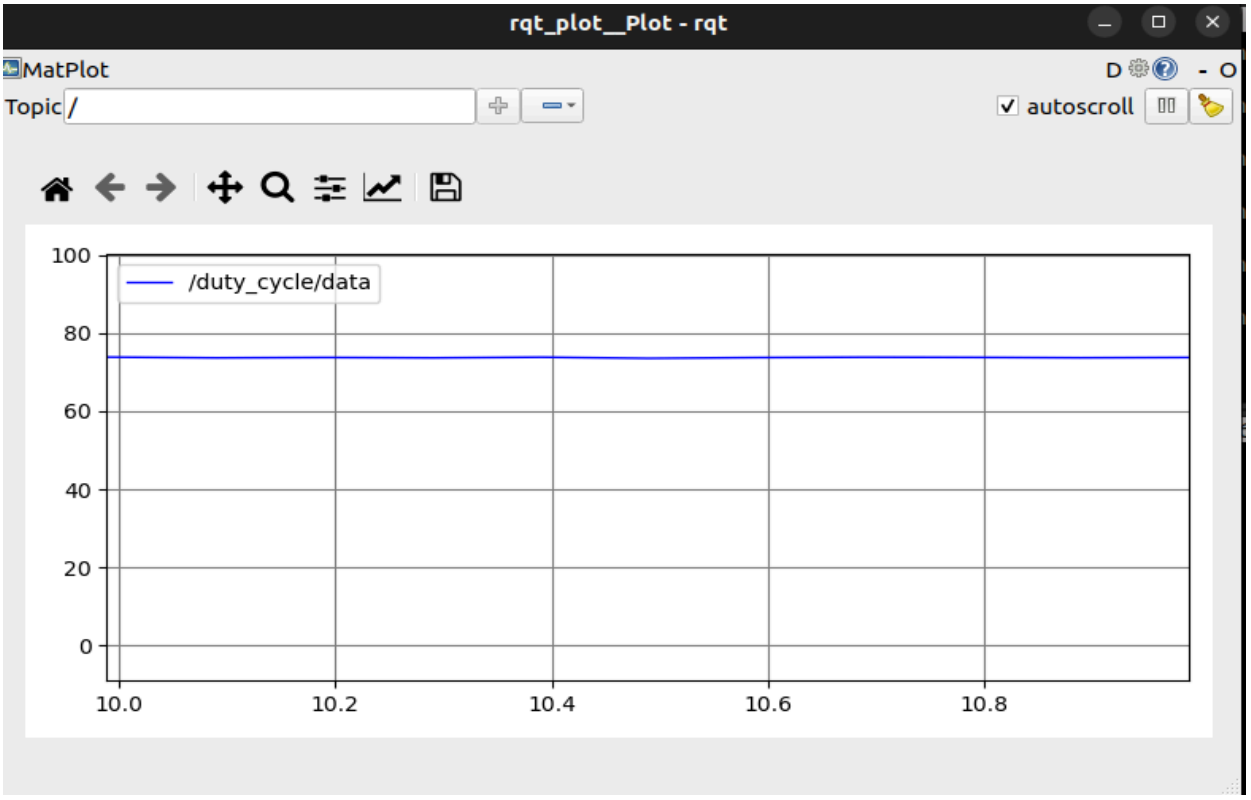
En la imagen se usa rqt_plot para monitorear el tópico /duty_cycle/data. La gráfica muestra cómo va cambiando el duty cycle del PWM conforme se mueve el potenciómetro en tiempo real. Se nota claramente cómo los valores suben y bajan de forma gradual, lo que confirma que la lectura analógica se está convirtiendo bien al ciclo de trabajo para controlar la velocidad del motor.

- Zona muerta del motor (data: 12.478632926940918)



En la imagen se observa la herramienta rqt_plot graficando el tópico /duty_cycle/data. La señal se mantiene cercana a un valor de 12.47, mostrando una variación mínima a lo largo del tiempo. Este comportamiento indica la zona muerta del motor, donde el ciclo de trabajo del PWM es insuficiente para generar movimiento en el motor.

- Gráfica valor estable del potenciómetro



En la imagen se muestra la herramienta rqt_plot monitoreando el tópico /duty_cycle/data. Se observa que la señal se mantiene constante en un valor aproximado de 74, formando una línea horizontal sin oscilaciones. Este comportamiento indica que el potenciómetro se dejó en una posición fija, entregando un ciclo de trabajo estable para mantener el motor a una velocidad continua.

Posterior al análisis de los resultados obtenidos, se puede concluir que los objetivos del proyecto se cumplieron de manera satisfactoria. En particular:

- Se implementó un sistema de control distribuido utilizando micro-ROS, donde el ESP32 funciona como nodo dentro del ecosistema ROS 2, permitiendo regular la velocidad y el sentido de giro del motor DC.
- Se configuró correctamente el entorno de trabajo uros_ws y el agente de micro-ROS, estableciendo una comunicación serial estable a 115200 baudios entre la computadora y el microcontrolador.

- Se migró la lógica de control convencional hacia una arquitectura basada en Publishers y Subscribers, permitiendo publicar variables como voltaje, duty cycle y estado del motor, mientras que los comandos de dirección se reciben mediante suscripción.
- Se corrigió el error de resolución en la conversión ADC–PWM, ajustando el mapeo al rango real del ESP32 (0–4095), lo que eliminó las señales erráticas observadas previamente.
- Finalmente, se validó la comunicación bidireccional mediante el envío de los comandos “D”, “I” y “S”, confirmando el correcto control del motor y la publicación de datos en ROS 2.

Conclusiones

El desarrollo de este proyecto permitió confirmar que los objetivos planteados se cumplieron correctamente. Se logró montar un sistema de control distribuido donde el ESP32 actúa como un nodo de ROS 2 mediante micro-ROS, lo que hace posible gestionar la dirección del motor DC y, al mismo tiempo, publicar datos en tiempo real como el voltaje y el ciclo de trabajo. Esta comunicación entre la PC y el microcontrolador resultó ser bastante estable tras configurar el agente de micro-ROS a 115200 baudios, algo que se validó enviando comandos por terminal y revisando la respuesta inmediata del hardware.

Un punto crucial en el proceso fue corregir el mapeo de la lectura analógica. Al ajustar la resolución del ADC al rango nativo del ESP32 (de 0 a 4095), se eliminaron los saltos bruscos y las señales erráticas que aparecían al inicio de las pruebas. Esto permitió que el motor responda de forma mucho más lineal al movimiento del potenciómetro, asegurando que el ciclo de trabajo aplicado coincida con lo que se espera físicamente en el sistema.

Por otro lado, el uso de herramientas como rqt_plot fue clave para analizar el comportamiento dinámico del control. Gracias a las gráficas, se pudo identificar con claridad la zona muerta del motor y la estabilidad de la señal bajo distintas condiciones. Como paso siguiente, el proyecto queda abierto para instalar sensores de realimentación, como un encoder, y así pasar de este esquema de lazo abierto a un control de velocidad mucho más preciso en lazo cerrado.

Bibliografía

Open Robotics. (2024). ROS 2 Documentation: Foxy, Humble, and Rolling. Recuperado de https://docs.ros.org/en/humble/index.html

Macenski, S. (2022). Modern Robotics with ROS 2: A practical approach to building robots with ROS 2. Packt Publishing.

Quigley, M., Gerkey, B., & Smart, W. D. (2015). Programming Robots with ROS: A Practical Introduction to the Robot Operating System. O'Reilly Media. (Nota: Aunque se centra en ROS 1, los capítulos sobre la filosofía de nodos y tópicos son fundamentales para entender la transición a ROS 2).

Angerer, C., & Hoffmann, A. (2020). Robot Operating System 2 (ROS 2): The Complete Guide. Springer Nature.